

ELEC5803 HLS Exam Study

Step-by-Step Solution to Question 3

Course-slide basis

This solution uses the same HLS ideas emphasized in:

- **Slides 5:** operation scheduling and resource-constrained scheduling.
- **Slides 6:** datapath allocation, bus-oriented interconnection, and hardware sharing.

1. Interpreting the neural network

The figure shows three computational stages of neurons:

$$(N_1, N_2, N_3) \rightarrow (N_4, N_5, N_6) \rightarrow (N_7, N_8, N_9) \rightarrow (O_1, O_2, O_3).$$

For a representative neuron such as N_4 , the inputs are the outputs of N_1 , N_2 , and N_3 :

$$s_4 = W_{14}N_1 + W_{24}N_2 + W_{34}N_3$$

and the neuron output is

$$N_4 = f(s_4), \quad f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0. \end{cases}$$

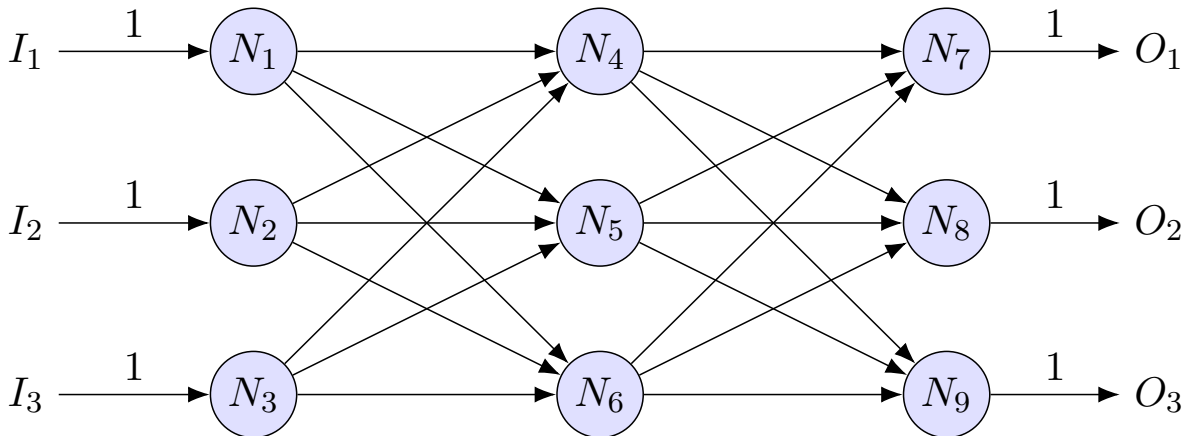


Figure 1: Native-Latex redraw of the Q3 network.

2. Part (a): scheduling diagram for one neuron

2.1 Operations of neuron N_4

For one neuron with three inputs, define:

$$\begin{aligned} m_1 &= W_{14}N_1, \\ m_2 &= W_{24}N_2, \\ m_3 &= W_{34}N_3, \\ a_1 &= m_1 + m_2, \\ a_2 &= a_1 + m_3, \\ N_4 &= f(a_2). \end{aligned}$$

So the computation graph is:

$$\{m_1, m_2, m_3\} \rightarrow a_1 \rightarrow a_2 \rightarrow f.$$

2.2 ASAP-style schedule

With unconstrained resources, the natural schedule is:

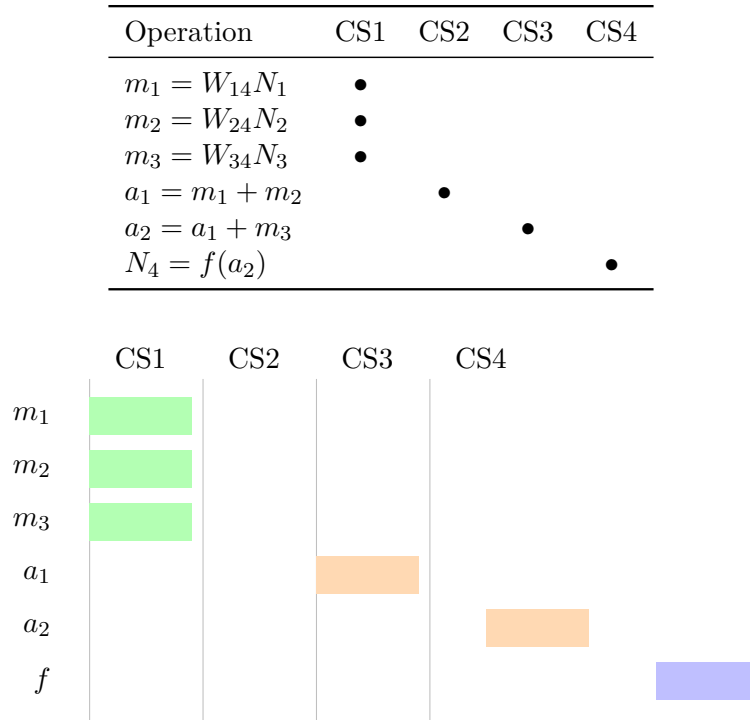


Figure 2: Scheduling diagram for a single neuron under unlimited resources.

2.3 Interpretation

This is exactly the kind of precedence-based scheduling discussed in Slides 5:

- the three multiplications are mutually independent,
- the first addition must wait for m_1 and m_2 ,
- the second addition must wait for a_1 and m_3 ,
- the activation function waits for the final sum.

3. Part (b): single-neuron hardware with at most two adders and two multipliers using a single bus

3.1 Resource constraint

We are allowed:

- at most 2 multipliers,
- at most 2 adders,
- a single shared bus.

The datapath for one neuron can therefore contain:

- an input/weight memory or register file,
- one shared single bus,
- two multiplier input register pairs,
- two multipliers,
- one accumulator adder for the partial sums,
- one threshold unit implementing the Heaviside function,
- a controller.

3.2 Single-bus schedule

Because the bus is shared, operand transfers must be serialized. A clean schedule is:

Step	Micro-operation
1	$\text{Bus} \leftarrow N_1$, load multiplier-1 operand register
2	$\text{Bus} \leftarrow W_{14}$, load multiplier-1 weight register
3	$\text{Bus} \leftarrow N_2$, load multiplier-2 operand register
4	$\text{Bus} \leftarrow W_{24}$, load multiplier-2 weight register
5	In parallel: $m_1 = W_{14}N_1$, $m_2 = W_{24}N_2$
6	$s_{12} = m_1 + m_2$
7	$\text{Bus} \leftarrow N_3$, load multiplier-1 operand register
8	$\text{Bus} \leftarrow W_{34}$, load multiplier-1 weight register
9	$m_3 = W_{34}N_3$
10	$s = s_{12} + m_3$
11	$N_4 = f(s)$

Only one adder is actually required for the accumulation chain. The second available adder does not reduce the latency here because the second addition depends on the first partial sum.

3.3 Hardware schematic

3.4 Why this satisfies the question

This design matches the HLS allocation ideas from Slides 6:

- arithmetic resources are bounded,
- the bus is a shared interconnection unit,

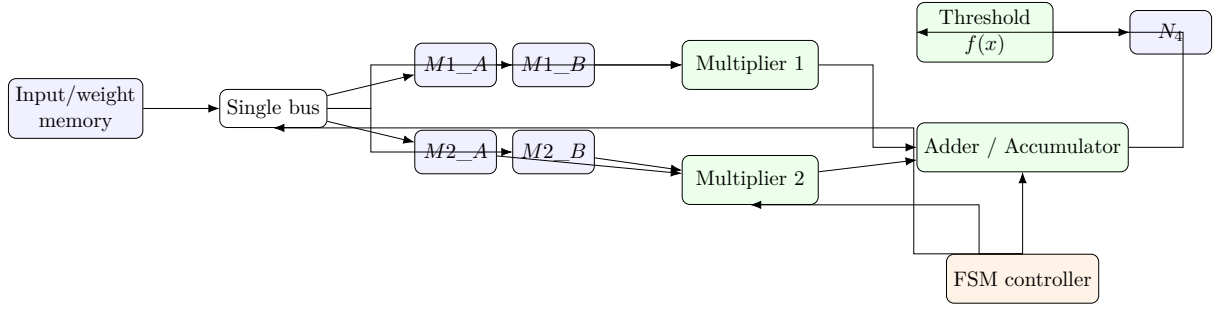


Figure 3: Single-neuron hardware under the given resource constraint and a single-bus datapath.

- the controller sequences operand transfers and functional-unit use,
- the datapath is a small FSMD-style realization of one neuron.

4. Part (c): total inference delay using the neuron design and a flexible multi-bus network

4.1 Assumptions

To keep the symbolic delay expression consistent with the question, use the following assumptions:

- memory access delay is T_{MEM} ,
- multiplication delay is T_{MUL} ,
- addition delay is T_{ADD} ,
- the threshold comparison is small and is not parameterized in the question, so it is neglected,
- the edges marked with weight 1 at the input and output are treated as direct connections rather than full multiply operations.

4.2 Delay of one 3-input neuron with two multipliers

With a *flexible multi-bus* implementation, bus conflicts are removed, so the neuron can use both multipliers efficiently.

One good schedule for a neuron is:

Stage	Operations	Delay
1	Fetch two input-weight pairs and compute m_1, m_2 in parallel	$T_{\text{MEM}} + T_{\text{MUL}}$
2	Fetch third pair, compute m_3 while also computing $a_1 = m_1 + m_2$	$\max(T_{\text{ADD}}, T_{\text{MEM}} + T_{\text{MUL}})$
3	Compute $a_2 = a_1 + m_3$	T_{ADD}

Therefore, the latency of one 3-input neuron is

$$T_{\text{neuron}} = (T_{\text{MEM}} + T_{\text{MUL}}) + \max(T_{\text{ADD}}, T_{\text{MEM}} + T_{\text{MUL}}) + T_{\text{ADD}}.$$

4.3 Layer-by-layer inference

In the given network:

- neurons N_4, N_5, N_6 form one 3-neuron layer,
- neurons N_7, N_8, N_9 form the next 3-neuron layer,
- all neurons in the same layer are independent and can therefore be evaluated in parallel in a multi-bus implementation.

Hence:

- delay of layer (N_4, N_5, N_6) is T_{neuron} ,
- delay of layer (N_7, N_8, N_9) is also T_{neuron} .

So the total inference delay is

$$T_{\text{inf}} = 2 T_{\text{neuron}}.$$

Substituting the neuron latency:

$$T_{\text{inf}} = 2 \left[(T_{\text{MEM}} + T_{\text{MUL}}) \max(T_{\text{ADD}}, T_{\text{MEM}} + T_{\text{MUL}}) T_{\text{ADD}} \right].$$

4.4 Common simplification

In most practical datapaths,

$$T_{\text{MEM}} + T_{\text{MUL}} \geq T_{\text{ADD}}.$$

Then the expression simplifies to:

$$T_{\text{inf}} = 2(2T_{\text{MEM}} + 2T_{\text{MUL}} + T_{\text{ADD}}).$$

4.5 Network schematic for inference

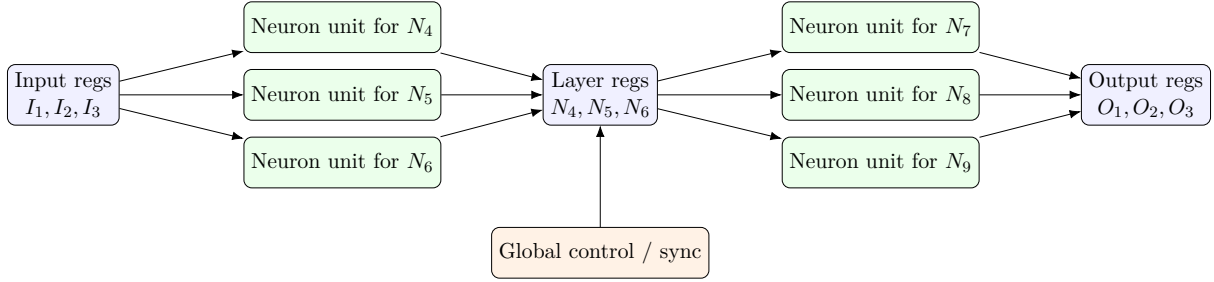


Figure 4: Flexible multi-bus network implementation: all three neurons in a layer execute in parallel, and the next layer starts after the layer registers are updated.

5. Final exam-style answers

(a) Scheduling diagram

For one neuron such as N_4 :

- CS1: three multiplications,
- CS2: first addition,
- CS3: second addition,
- CS4: Heaviside activation.

(b) Single-bus neuron hardware

Use a small FSMD datapath with:

- one input/weight memory,
- one shared bus,
- two multipliers,
- one accumulator adder,
- one threshold unit,
- one controller.

Serialized bus transfers load the two multipliers, then the partial sums are accumulated.

(c) Total inference delay

Using a flexible multi-bus network so that all neurons in a layer run in parallel:

$$T_{\text{inf}} = 2 \left[(T_{\text{MEM}} + T_{\text{MUL}}) \max(T_{\text{ADD}}, T_{\text{MEM}} + T_{\text{MUL}}) T_{\text{ADD}} \right].$$

If $T_{\text{MEM}} + T_{\text{MUL}} \geq T_{\text{ADD}}$, then

$$T_{\text{inf}} = 2 (2T_{\text{MEM}} + 2T_{\text{MUL}} + T_{\text{ADD}}).$$